CO3 – Assignment 3

Constructors, Destructors and Operator Overloading in C++

Name	Hemalatha.C
Register Number	192524475
GitHub Repository	https://github.com/Hemalatha-1806/DSA0102-C-
Compiler used	g++ 13.3.0 (g++ -std=c++17 -Wall -Wextra)

Contents

Q	Topic	Marks
1	Constructors and constructor overloading (Library Book Records)	6
2	Copy constructors: shallow and deep copy (Hospital Prescription Records)	7
3	Dynamic constructors and destructors (Auditorium Seat Booking)	7
4	Arithmetic and stream operators (Money for a Billing Counter)	7
5	Increment, comparison and subscript operators (Playlist and Track Timer)	6
6	The rules of operator overloading (SMS Text class)	7
7	Complex-number class for an AC impedance calculator	10

Q1 – Constructors and Constructor Overloading

Problem: a library creates Book objects in three ways – full details, donated book (title only) and an empty record. No Book may ever have a negative (garbage) number of copies.

1. Implementation

One "main" constructor does the real work and the only validation. The other two constructors delegate to it, so the code is written once. Every constructor uses a member initializer list.

book.cpp

```cpp
#include <iostream>
#include <string>
#include <stdexcept>

class Book {
    std::string title;
    std::string author;
    int copies;

    // single place where the number of copies is checked
    static int checked(int c) {
        if (c < 0) throw std::invalid_argument("number of copies cannot be negative");
        return c;
    }

public:
    // main constructor: does the real work and the validation
    Book(const std::string& t, const std::string& a, int c)
        : title(t), author(a), copies(checked(c)) {}

    // donated book: only a title is known (delegates)
    explicit Book(const std::string& t) : Book(t, "Unknown", 1) {}

    // empty record to be filled later (delegates)
    Book() : Book("Untitled", "Unknown", 0) {}

    friend std::ostream& operator<<(std::ostream& os, const Book& b) {
        return os << "[" << b.title << " | " << b.author << " | copies=" << b.copies << "]";
    }
};

int main() {
    Book b1;                                // default
    Book b2("The Hobbit", "J.R.R. Tolkien", 5); // full details
    Book b3("Wings of Fire");               // donated, title only
    std::cout << b1 << '\n' << b2 << '\n' << b3 << '\n';

    try {
        Book bad("Broken Book", "Nobody", -3);
        std::cout << bad << '\n';
    } catch (const std::invalid_argument& e) {
        std::cout << "Rejected: " << e.what() << '\n';
    }
}
```

OUTPUT (g++ -std=c++17 -Wall -Wextra book.cpp)

```
[Untitled | Unknown | copies=0]
[The Hobbit | J.R.R. Tolkien | copies=5]
[Wings of Fire | Unknown | copies=1]
Rejected: number of copies cannot be negative
```

A negative number of copies is rejected by throwing an exception from the initializer list, so a half-built Book never exists. Because all three constructors end up in the main constructor, there is no way around the check.

2. Design & Analysis

When does the compiler give a default constructor?

The compiler supplies a default constructor **only when the class has no constructor of its own**. The moment you write any constructor (even a parameterised one), the compiler stops supplying it. If you still need Book b; or Book shelf[3]; you must write Book() {} yourself (or use = default).

Why initializer lists are better than assignment in the body

- Members are created **before** the constructor body runs. If you assign inside the body, the member is first default-constructed and then assigned – two steps instead of one (a waste for std::string).
- const members, reference members and members whose class has no default constructor **can only be initialised** in the list.
- It is also the only way to call a base-class constructor or a delegating constructor.

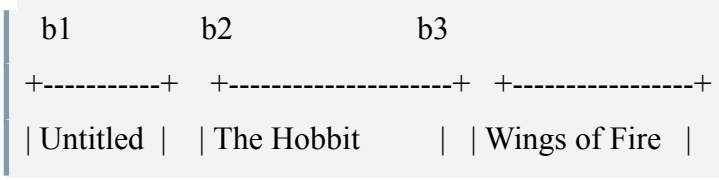
Order of initialisation

Members are initialised in the order they are **declared in the class**, not the order written in the list. The compiler warns about this with -Wreorder. Experiment below.

State of three objects after construction

Object	Created with	title	author	copies
b1	Book b1;	"Untitled"	"Unknown"	0
b2	Book b2("The Hobbit","J.R.R. Tolkien",5);	"The Hobbit"	"J.R.R. Tolkien"	5
b3	Book b3("Wings of Fire");	"Wings of Fire"	"Unknown"	1

STATE DIAGRAM



```
| Unknown   |   | J.R.R. Tolkien     |   | Unknown        |
| 0         |   | 5                  |   | 1              |
+-----------+   +--------------------+   +----------------+
```

3. Experimental Investigation

(a) Every constructor used and printed

See the output in section 1 – b1 (default), b2 (full), b3 (title only).

(b) Default constructor removed, array of Books created

noDefault.cpp

```cpp
#include <string>
class Book {
    std::string title; int copies;
public:
    Book(const std::string& t, int c) : title(t), copies(c) {}
};
int main() { Book shelf[3]; }

```

COMPILER ERROR

```
noDefault.cpp: In function 'int main()':
noDefault.cpp:7:26: error: no matching function for call to 'Book::Book()'
    7 | int main() { Book shelf[3]; }
noDefault.cpp:5:5: note: candidate: 'Book::Book(const std::string&, int)'
noDefault.cpp:5:5: note:   candidate expects 2 arguments, 0 provided
```

Each of the 3 array elements needs a no-argument constructor, and the compiler did not generate one because another constructor exists.

(c) Wrong initialisation order and -Wreorder

reorder.cpp

```cpp
#include <iostream>
struct Demo {
    int a;
    int b;
    Demo(int x) : b(x), a(b + 1) {}   // list order: b, a   declaration order: a, b
};
int main() {
    Demo d(10);
    std::cout << "a=" << d.a << " b=" << d.b << '\n';   // a is garbage: b was not set yet
}

```

WARNING + RUN (shortened)

```
reorder.cpp:4:9: warning: 'Demo::b' will be initialized after [-Wreorder]
reorder.cpp:5:5: warning:   when initialized here [-Wreorder]
reorder.cpp:5:27: warning: member 'Demo::b' is used uninitialized [-Wuninitialized]
a=1 b=10
```

a is declared first, so it is initialised first – using b which has no value yet. The printed a=1 is garbage (it can be any value on another machine). Fix: write the list in declaration order or do not use one member to initialise another.

(d) Negative number of copies

Book bad("Broken Book","Nobody",-3); printed Rejected: number of copies cannot be negative (see section 1). The object was never created.

4. Design Question – when should a constructor be explicit?

Mark a constructor explicit when it can be called with a single argument and you do **not** want the compiler to silently convert that argument into your class. The title-only Book constructor is a good example: a plain string should not quietly become a Book.

explicit.cpp (explicit commented out – COMPILES)

```cpp
#include <iostream>
#include <string>
struct Book {
    std::string title;
    explicit Book(const std::string& t) : title(t) {}
};
void shelve(const Book& b) { std::cout << "Shelved: " << b.title << '\n'; }
int main() {
    shelve(std::string("Gitanjali"));   // implicit conversion string -> Book
}
```

OUTPUT without explicit

```
Shelved: Gitanjali
```

Now remove the comment so the constructor is explicit Book(const std::string& t):

COMPILER ERROR with explicit

```
explicit.cpp:9:17: error: invalid initialization of reference of type 'const Book&'
        from expression of type 'std::string'
  9 |     shelve(std::string("Gitanjali"));
```

With explicit the programmer must write shelve(Book("Gitanjali")), which makes the conversion visible.

5. Alternative Concept and Complexity Analysis

Alternative: one constructor with default arguments instead of three overloaded constructors with delegation.

defaultargs.cpp (class part)

```cpp
#include <iostream>
#include <string>
#include <stdexcept>
class Book {
    std::string title, author; int copies;
public:
    Book(const std::string& t = "Untitled", const std::string& a = "Unknown", int c = 0)
        : title(t), author(a), copies(c < 0 ? throw std::invalid_argument("neg") : c) {}
    friend std::ostream& operator<<(std::ostream& o, const Book& b){return o<<b.title<<"/"<<b.author<<"/"<<b.copies;}
};
int main(){ Book a, b("Title"), c("T","A",4); std::cout<<a<<'\n'<<b<<'\n'<<c<<'\n'; }
```

OUTPUT

```
Untitled/Unknown/0
Title/Unknown/0
T/A/4
```

	Overloads + delegation	One constructor with default arguments
Time of object creation	O(1)	O(1)
Space per object	O(1) (title, author, int)	O(1) – same
Constructors to write	3	1
Places where copies is checked	1 (inside the main constructor / checked())	1 (the only constructor)
Weak point	More lines	Book("Title") gives copies = 0 not 1; cannot make only the title-only call explicit; defaults are hidden in the header

Conclusion: both are O(1) in time and space. The default-argument design has the validation in one constructor, so it is the fewest places (one). The delegation design also validates in one place, but gives different default values for different situations (donated book gets 1 copy, empty record gets 0), which is clearer for this library problem. I would keep the overloads + delegation.

Q2 – Copy Constructors: Shallow and Deep Copies

Problem: a Prescription owns a dynamically allocated array of medicine names. Copying it for a follow-up visit changes the original and the program crashes at the end.

1. Step one – the class WITHOUT a copy constructor (the bug)

shallow.cpp

```cpp
// Version 1: NO copy constructor, NO copy assignment  (compiler copies the pointer only)
#include <iostream>
#include <string>
class Prescription {
    int n;
    std::string* medicines;
public:
    explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
    ~Prescription() { delete[] medicines; }
    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};
int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;          // shallow copy: q.medicines == p.medicines
    q.set(1, "Cetirizine");      // changes p as well!
    std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1) << '\n';
}   // q and p both delete[] the same block -> double free

```

OUTPUT

```
Paracetamol Cetirizine | Paracetamol Cetirizine
Segmentation fault   (exit code 139)
```

Two problems are visible. (1) p should still hold "ORS" in position 1, but it now shows "Cetirizine" – changing q changed p. (2) The program crashes when it ends.

Sanitizer report (g++ -std=c++17 -g -fsanitize=address shallow.cpp)

ASAN OUTPUT (shortened)

```
==ERROR: AddressSanitizer: heap-use-after-free on address 0x507000000090
READ of size 8 at 0x507000000090 thread T0
    #0 ... in Prescription::~Prescription() shallow.cpp:9
    #1 ... in main shallow.cpp:20

0x507000000090 is located 0 bytes inside of 72-byte region
freed by thread T0 here:
    #0 ... in operator delete[](void*, unsigned long)
```

```
  #1 ... in Prescription::~Prescription() shallow.cpp:9
  #2 ... in main shallow.cpp:20

previously allocated by thread T0 here:
  #0 ... in operator new[](unsigned long)
  #1 ... in Prescription::Prescription(int) shallow.cpp:8
  #2 ... in main shallow.cpp:14
```

Reading it: the memory block was allocated once in the constructor (line 14, for p), freed by the first destructor, and then the second destructor touched/freed the same block again.

2. Step two – the fixed class (Rule of Three)

deep.cpp (TODO 1, 2, 3 completed)

```cpp
// Version 2: Rule of Three implemented
#include <iostream>
#include <string>

class Prescription {
    int n;
    std::string* medicines;   // dynamically allocated array

public:
    explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}

    // TODO 1: deep-copy constructor
    Prescription(const Prescription& other)
        : n(other.n), medicines(new std::string[other.n]) {
        for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
    }

    // TODO 2: copy assignment
    Prescription& operator=(const Prescription& other) {
        if (this == &other) return *this;            // self-assignment guard
        std::string* fresh = new std::string[other.n]; // build new array first
        for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
        delete[] medicines;                          // release old array
        medicines = fresh;
        n = other.n;
        return *this;
    }

    ~Prescription()
    {
        // TODO 3
        delete[] medicines;
    }

    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};

int main()
{
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p; // copy constructor
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p; // copy assignment
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1)
              << " | " << r.get(0) << ' ' << r.get(1) << '\n';
}

```

OUTPUT

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

This matches the expected output exactly. Running with -fsanitize=address printed **no errors** (clean exit).

Copy constructor removed (as the question asks): I deleted TODO 1 and kept the assignment operator and destructor. The compiler then generates a shallow copy for Prescription q = p; and ASan reports the same error as before:

ASAN OUTPUT – copy constructor missing

```
ERROR: AddressSanitizer: heap-use-after-free on address 0x507000000090
READ of size 8 ... #0 in Prescription::~Prescription() nocopyctor.cpp:26
                #1 in main nocopyctor.cpp:45
```

3. Design & Analysis

Memory diagrams

DIAGRAM

```
SHALLOW COPY  (compiler default: copies the pointer only)

 p:  n=2  medicines --+
                      +--->  [ "Paracetamol" | "Cetirizine" ]   <- ONE array
 q:  n=2  medicines --+
 (set on q changes p;  both destructors delete[] the same array)



DEEP COPY  (our copy constructor)

 p:  n=2  medicines ----->  [ "Paracetamol" | "ORS"        ]
 q:  n=2  medicines ----->  [ "Paracetamol" | "Cetirizine" ]   <- own array
 (each object deletes only its own array)
```

Why the shallow version crashes (double delete)

The compiler-made copy constructor copies every member one by one, so the pointer value is copied but not what it points to. At the end of main, q is destroyed first and runs delete[] medicines. Then p is destroyed and runs delete[] on the same address, which is already freed – a double delete (undefined behaviour). Here it showed up as a crash.

Rule of Three

If a class needs a **user-written destructor, copy constructor or copy assignment operator**, it almost certainly needs all three. They go together because all three deal with the same thing: who owns the dynamic memory. The destructor frees it, the copy constructor makes a new owned copy, and the assignment operator replaces the old owned memory with a new copy. If one is missing, two objects end up owning the same block.

Three situations where the copy constructor is called

- Creating an object from an existing object: Prescription q = p; or Prescription q(p);
- Passing an object **by value** to a function: void f(Prescription x)
- Returning an object **by value** from a function (unless the compiler removes the copy by copy elision).

4. Experimental Investigation

(a) How many copies – by value vs const reference

experiments.cpp (part)

```
28   void byValue(Prescription p)           { (void)p; }
29   void byConstRef(const Prescription& p) { (void)p; }
```

OUTPUT

```
Calling byValue:
  [copy constructor called]
Calling byConstRef:
Total copies = 1
```

Passing by value made **1 copy**; passing by const reference made **0 copies**.

(b) Time to copy 10, 1,000 and 100,000 medicines

Medicines (n)	Time of one deep copy (my run, -O0)
10	0.31 microseconds
1,000	20.0 microseconds
100,000	1,643.6 microseconds (1.6 ms)

When n grows 100 times (1,000 → 100,000) the time grows about 82 times, so the copy cost is **linear, O(n)**. Your numbers will be different on your computer, but the pattern should be the same. Run it 3–5 times and report the average.

(c) Sanitizer results

Version	ASan result
Without copy constructor	heap-use-after-free in the destructor (double delete)
With deep copy + assignment + destructor	No error reported

(d) Self-assignment (p = p)

OUTPUT

```
Self assignment test:
  [self-assignment ignored]
ABC (data still intact)
```

The guard if (this == &other) return *this; stops the object from deleting its own array before copying from it. Without the guard, delete[] medicines would destroy the data that we are about to copy.

5. Conclusion

You can rely on the compiler-generated copy constructor when every member copies itself correctly – ints, doubles, std::string, std::vector, smart pointers. You must write your own when the class **owns a raw resource** (a new-ed array, a file handle, a socket), because the default copy would only copy the pointer.

6. Alternative Concept and Complexity Analysis

Alternative: store the medicines in a std::vector<std::string>. Then no copy constructor, assignment or destructor is needed (Rule of Zero) – the vector already makes a deep copy.

ALTERNATIVE

```
class Prescription {
    std::vector<std::string> medicines;
public:
    explicit Prescription(int count) : medicines(count) {}
```

```
    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};   // compiler-generated copy, assignment and destructor are all correct
```

Design	Time to copy n medicines	Extra space	Safe?
Raw array + own deep copy	O(n)	O(n)	Yes, if all 3 functions are correct
std::vector (Rule of Zero)	O(n)	O(n)	Yes, automatically
Shallow copy (compiler default for raw pointer)	O(1)	O(1)	No – shared array, double delete

Both correct designs cost the same, O(n) time and O(n) space. The vector version just has much less code and fewer chances for mistakes. The shallow copy is faster only because it does not really copy, and that is exactly why it is wrong.

Q3 – Dynamic Constructors and Destructors

Problem: each event has its own number of rows and seats per row. The seat map is created with new when the event is made, and everything must be released (with a closing summary) when the event is destroyed.

1. Implementation

auditorium.cpp

```cpp
#include <iostream>
#include <string>
#include <stdexcept>

class Event {
    std::string name;
    int rows, seats;
    char** map;           // array of row pointers, each row allocated with new
    int booked;

public:
    // dynamic constructor: size is known only at run time
    Event(const std::string& n, int r, int s)
        : name(n), rows(r), seats(s), map(nullptr), booked(0) {
        if (r < 0 || s < 0) throw std::invalid_argument("rows/seats cannot be negative");
        if (r > 0) {
            map = new char*[r];
            for (int i = 0; i < r; ++i) {
                map[i] = new char[s];
                for (int j = 0; j < s; ++j) map[i][j] = 'O';    // O = open, X = booked
            }
        }
        std::cout << "[ctor] " << name << " created (" << rows << " x " << seats << ")\n";
    }

    // copying would double-delete, so it is disabled
    Event(const Event&) = delete;
    Event& operator=(const Event&) = delete;

    bool book(int r, int s) {
        if (r < 0 || r >= rows || s < 0 || s >= seats || map[r][s] == 'X') return false;
        map[r][s] = 'X'; ++booked; return true;
    }
    bool cancel(int r, int s) {
        if (r < 0 || r >= rows || s < 0 || s >= seats || map[r][s] != 'X') return false;
        map[r][s] = 'O'; --booked; return true;
    }
    void show() const {
        for (int i = 0; i < rows; ++i) {
            for (int j = 0; j < seats; ++j) std::cout << map[i][j] << ' ';
            std::cout << '\n';
        }
    }

    ~Event() {
        for (int i = 0; i < rows; ++i) delete[] map[i];    // every row
        delete[] map;                                      // then the row-pointer array
        std::cout << "[dtor] " << name << " closed. Booked seats at close: " << booked
                  << " of " << rows * seats << '\n';
    }
};

int main() {
    Event stackEvent("Stack Concert", 3, 4);
    stackEvent.book(0, 1); stackEvent.book(2, 3); stackEvent.book(0, 1);   // 2nd is rejected
    stackEvent.cancel(2, 3);
    stackEvent.show();

    {
        Event blockEvent("Block Seminar", 2, 2);
        blockEvent.book(1, 1);
    }   // destroyed here

    Event* heapEvent = new Event("Heap Drama", 2, 3);
    heapEvent->book(0, 0);
    delete heapEvent;

    Event empty("Zero Row Event", 0, 10);
    std::cout << "book on empty event: " << std::boolalpha << empty.book(0, 0) << '\n';
    std::cout << "end of main\n";
}
```

OUTPUT (compiled with -fsanitize=address – no errors)

```
[ctor] Stack Concert created (3 x 4)
O X O O
```

```
O O O O
O O O O
[ctor] Block Seminar created (2 x 2)
[dtor] Block Seminar closed. Booked seats at close: 1 of 4
[ctor] Heap Drama created (2 x 3)
[dtor] Heap Drama closed. Booked seats at close: 1 of 6
[ctor] Zero Row Event created (0 x 10)
book on empty event: false
end of main
[dtor] Zero Row Event closed. Booked seats at close: 0 of 0
[dtor] Stack Concert closed. Booked seats at close: 1 of 12
```

Booking seat (0,1) twice – the second attempt is rejected. Seat (2,3) was booked and then cancelled, so only one seat is left booked. The copy constructor and copy assignment are deleted on purpose: copying an Event would copy the pointers and cause the double delete we saw in Q2.

2. Design & Analysis

Order of constructors and destructors

Kind of object	Constructor runs	Destructor runs
Local (stack) object	When the declaration line is executed	At the closing brace of its scope (end of main) – in **reverse** order of creation
Block-scope object	When execution reaches its declaration inside the block	At the end of that block }
Heap object (new)	At the new expression	Only when delete is called – never automatically

In the output: "Zero Row Event" was created after "Stack Concert", so it is destroyed **before** it (last created, first destroyed).

What if delete is forgotten?

The destructor never runs, so the rows and the pointer array are never freed (a **memory leak**) and the closing summary is never printed. The memory is returned only when the whole program ends.

What if delete is used instead of delete[]?

When memory was allocated with new[], it must be released with delete[]. Using plain delete is **undefined behaviour**: only one element may be destroyed and the memory manager may get confused. ASan reports it as an alloc-dealloc-mismatch error.

Why does a destructor take no arguments and cannot be overloaded?

The compiler calls the destructor automatically at the end of an object's life – there is no place in the code where we could pass arguments. And since the call is automatic, there must be exactly one destructor per class, otherwise the compiler would not know which one to run. (It also has no return type.)

Memory layout of the seat map (3 rows × 4 seats)

DIAGRAM

```
 map (char**)
  |
  v
+--------+    +---+---+---+---+
| map[0] |---->| O | X | O | O |   row 0  (own new char[4])
+--------+    +---+---+---+---+
| map[1] |---->| O | O | O | O |   row 1
+--------+    +---+---+---+---+
| map[2] |---->| O | O | O | O |   row 2
+--------+    +---+---+---+---+
array of 3 row pointers (new char*[3])


Allocations = 1 (pointer array) + 3 (rows) = 4
Release order: delete[] map[0], map[1], map[2], then delete[] map
```

3. Experimental Investigation

(a) Order of messages for each kind of object

Taken from the output above:

Object	Created	Destroyed
Stack Concert (local)	1st	last (end of main)

Object	Created	Destroyed
Block Seminar (inside { })	2nd	immediately at the closing brace of the block
Heap Drama (new)	3rd	at the line delete heapEvent;
Zero Row Event (local)	4th	just before Stack Concert at the end of main

(b) Sanitizer: missing delete, then correct code

Test program with the delete line commented out:

leak.cpp

```cpp
#include <iostream>
struct Big { int* data; Big() : data(new int[100]) {} ~Big() { delete[] data; } };
int main() {
    Big* p = new Big();      // delete p;  <-- forgotten
    std::cout << "done\n";
}
```

Compile and run **on your own computer** with:

COMMAND

```
g++ -std=c++17 -g -fsanitize=address leak.cpp -o leak
./leak
```

Expected result: the program prints done and then the LeakSanitizer part of ASan reports something like ERROR: LeakSanitizer: detected memory leaks with Direct leak of 8 byte(s) (the Big object) and an Indirect leak of 400 byte(s) (the int array), and points to the new line. Paste your own report in your document and screenshot it. (I could not get LeakSanitizer to run inside the environment I used to prepare this, so I am not copying a made-up report here.)

With the correct code (auditorium.cpp above, with delete heapEvent;) ASan printed no errors at all.

(c) Memory used by seat maps

Seat map	Array of row pointers (this design)	Allocations
10 × 20	80 B pointers + 200 B seats = 280 B	11
100 × 200	800 B + 20,000 B = 20,800 B	101
1,000 × 2,000	8,000 B + 2,000,000 B = 2,008,000 B (about 1.9 MB)	1,001

(On a 64-bit system a pointer is 8 bytes and a char is 1 byte. The real allocator adds a small overhead to each allocation, so the real total is a bit higher for the many-row cases.)

(d) Event with zero rows

Handled: the constructor skips the allocation (map stays nullptr), book(0,0) returns false because the row check fails first, and the destructor loop runs zero times and delete[] nullptr is safe. Output: book on empty event: false and closed ... 0 of 0.

4. Conclusion

With RAII (Resource Acquisition Is Initialization) the memory is obtained in the constructor and released in the destructor. Because the destructor runs automatically when the object goes out of scope – even if an exception is thrown or the function returns early – the programmer cannot forget to free it. Leaks and double-frees become much less likely, and the closing summary is guaranteed to print.

5. Predict the Output

predict.cpp

```cpp
#include <iostream>
#include <string>

struct Logger {
    std::string n;
    Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
    Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }
    ~Logger() { std::cout << '-' << n << ' '; }
};

void use(Logger) { std::cout << "use "; }

int main()
{
    Logger a("A");
    {
        Logger b("B");
    }
    use(a);
    Logger* p = new Logger("P");
    delete p;
    std::cout << "end ";
}

```

My predicted output (worked out before running):

```
PREDICTION
+A +B -B copyAc use -Ac +P -P end -A
```

Actual output after compiling and running:

```
ACTUAL OUTPUT
+A +B -B copyAc use -Ac +P -P end -A
```

Prediction and actual output are the same. Step by step:

Output	Reason
+A	Logger a("A") is constructed.
+B -B	b is created inside the inner block and destroyed at its closing brace.
copyAc	use(a) takes the parameter **by value**, so the copy constructor runs; the copy's name is "A" + "c" = "Ac".
use	The function body runs.
-Ac	The copy (parameter) is destroyed when the call finishes – the original a is not affected.
+P -P	A heap object is created by new and destroyed at once by delete p.
end	Printed at the end of main's body.
-A	The local a is destroyed when main ends.

Note: whether the parameter copy is destroyed before or after use returns control is decided by the compiler (GCC destroys it at the end of the full call expression), but in both cases it happens before +P, so the visible output is the same. If you were to pass a by const reference (void use(const Logger&)), the copy and -Ac would disappear.

6. Alternative Concept and Complexity Analysis

Alternative: store the seats in one block of rows × seats elements (index r * seats + s), or in std::vector<std::vector<char>>.

```
ALTERNATIVE
// one block
```

```
char* map = new char[rows * seats];
map[r * seats + s] = 'X';        // access

// destructor needs only one line
delete[] map;

// or: std::vector<char> map(rows * seats);   -> no destructor needed at all
```

Layout	Access time	Allocations	Extra space for pointers/objects
Array of row pointers (used)	O(1) (two lookups)	rows + 1	rows × 8 B (80 B / 800 B / 8,000 B)
One flat block	O(1) (one multiply-add)	1	none
vector<vector<char>>	O(1)	rows + 1	rows × 24 B (a vector object per row)

Access is O(1) in all three. The flat block needs only **one allocation** and one delete[], has no per-row pointers and keeps all seats together in memory (better for the cache). The pointer-array design is the most readable (map[r][s]) and lets rows have different lengths, but costs rows + 1 allocations. For a fixed-size rectangular seat map I would choose the flat block (or a single std::vector).

Q4 – Overloading Arithmetic and Stream Operators

Problem: the billing counter uses calls like add(a, b) and print(c), which are hard to read. A Money class storing paise in a long long lets us write a + b and std::cout << c.

1. Complete the Code (TODO 1 and TODO 2 done)

money.cpp

```cpp
#include <iomanip>
#include <iostream>

class Money {
    long long paise;

public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    // TODO 1: + as member, returns a new object
    Money operator+(const Money& o) const { Money r; r.paise = paise + o.paise; return r; }

    // extra operators for the billing counter
    Money operator-(const Money& o) const { Money r; r.paise = paise - o.paise; return r; }
    Money& operator+=(const Money& o) { paise += o.paise; return *this; }
    bool operator<(const Money& o) const { return paise < o.paise; }

    // TODO 2: == as member, << as friend
    bool operator==(const Money& o) const { return paise == o.paise; }

    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        long long v = m.paise < 0 ? -m.paise : m.paise;
        if (m.paise < 0) os << "-";
        return os << "Rs " << v / 100 << '.' << std::setw(2) << std::setfill('0') << v % 100
                  << std::setfill(' ');
    }
    friend std::istream& operator>>(std::istream& is, Money& m) {   // reads: rupees paise
        long long r; int p;
        if (is >> r >> p) m = Money(r, p);
        return is;
    }
};

int main()
{
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';
}
```

OUTPUT (matches the expected output)

```
Rs 200.00
true
```

Money is kept as a whole number of paise, so there is no rounding error. << prints the rupees, a dot, and the paise padded to two digits (std::setw(2) with setfill('0')), and it handles negative amounts by printing the minus sign first.

2. Design & Analysis

Why + returns a new object but += returns a reference

a + b must **not** change a or b, so it builds and returns a new Money. a += b is meant to change a; it returns *this by reference so that no copy is made and chaining such as a += b += c works.

Members and friends

Operator	Member / friend	Reason
+, -, <, ==	Member (const)	Left operand is a Money; they only read paise.
+=	Member	It changes the left object, so it belongs to the object.
<<, >>	Friend (non-member)	Left operand is std::ostream/std::istream, a class we cannot modify. Friend gives access to private paise.

Why << must be a non-member

For a member operator the left operand is the object itself (*this). In std::cout << c the left operand is std::cout, not a Money, so it would have to be a member of std::ostream, which we cannot edit. So it is written as a free function operator<<(std::ostream&, const Money&), made a friend to reach the private data.

How the compiler translates a + b

For a member operator, a + b becomes a.operator+(b). For a non-member it becomes operator+(a, b). Likewise std::cout << c becomes operator<<(std::cout, c).

3. Experimental Investigation

tests.cpp (uses the class from money_class.h)

```cpp
#include <chrono>
#include <iomanip>
#include <iostream>
#include <sstream>
#include "money_class.h"

int main() {
    std::cout << "Carry test : " << Money(0, 75) << " + " << Money(0, 50) << " = "
              << Money(0, 75) + Money(0, 50) << '\n';
    std::cout << "Negative   : " << Money(10) << " - " << Money(25, 50) << " = "
              << Money(10) - Money(25, 50) << '\n';

    Money bill;                       // bill calculation with operators
    Money rice(120, 50), oil(210), tax(18, 75), discount(15);
    bill += rice; bill += oil; bill += tax;
    Money total = bill - discount;
    std::cout << "Bill       : " << rice << " + " << oil << " + " << tax << " - " << discount
              << " = " << total << '\n';
    std::cout << "total < Rs 400 ? " << std::boolalpha << (total < Money(400)) << '\n';

    std::istringstream in("45 30");
    Money x; in >> x;
    std::cout << "Read from stream: " << x << '\n';

    // 1,000,000 additions of Rs 0.10
    Money m; double d = 0.0;
    for (int i = 0; i < 1000000; ++i) m += Money(0, 10);
    for (int i = 0; i < 1000000; ++i) d += 0.10;
    std::cout << std::setprecision(17);
    std::cout << "Money  after 1,000,000 x 0.10 : " << m << '\n';
    std::cout << "double after 1,000,000 x 0.10 : " << d << '\n';
    std::cout << "double == 100000 ? " << (d == 100000.0) << '\n';
}

```

OUTPUT

```
Carry test : Rs 0.75 + Rs 0.50 = Rs 1.25
Negative   : Rs 10.00 - Rs 25.50 = -Rs 15.50
Bill       : Rs 120.50 + Rs 210.00 + Rs 18.75 - Rs 15.00 = Rs 334.25
total < Rs 400 ? true
Read from stream: Rs 45.30
Money  after 1,000,000 x 0.10 : Rs 100000.00
double after 1,000,000 x 0.10 : 100000.00000133288
double == 100000 ? false
```

(a) Carry of paise

75 + 50 = 125 paise, which is printed as Rs 1.25 – the carry from paise into rupees works automatically because everything is stored as paise.

(b) Negative result

Rs 10.00 − Rs 25.50 gives -Rs 15.50. My decision: allow a negative Money value (a refund, a credit note or a balance owed are real situations), but let the **billing code** check total < Money(0) where a negative bill makes no sense. The alternative is to throw an exception inside operator-, which is safer for amounts that can never be negative, but then refunds cannot be represented.

(c) Readability

COMPARISON

```
// without operator overloading
Money total = subtract(add(add(rice, oil), tax), discount);
print(total);

// with operator overloading
Money total = rice + oil + tax - discount;
std::cout << total;
```

The second version reads like the bill on paper, has fewer nested brackets, and is harder to get the order wrong.

(d) Money vs double for 1,000,000 additions of Rs 0.10

Money gives exactly **Rs 100000.00**. The double version gave **100000.00000133288** and d == 100000 is false. 0.10 cannot be stored exactly in binary, so every addition carries a tiny error and they pile up. This is why money should never be stored in a double.

4. Engineering Decision – should Money × Money be allowed?

No. Rs 5 × Rs 5 would be "25 rupee-squared", which has no meaning in billing. Operators that make sense and operators that do not:

Makes sense	Does not make sense
Money + Money, Money − Money	Money × Money

Makes sense	Does not make sense
Money × number (price × quantity), Money / number	Money + plain number (is 5 rupees or 5 paise?)
Money / Money → a plain number (ratio)	Money % Money for billing
==, !=, <, >, <=, >=, unary −, +=, −=	

5. Alternative Concept and Complexity Analysis

Alternative: write the operators as non-member friend functions, or use named functions add() and equals().

FRIEND VERSION

```
friend Money operator+(const Money& a, const Money& b) {
   Money m; m.paise = a.paise + b.paise; return m;
}
```

Time and space do not change: every operator does a few integer operations and stores one long long, so all are **O(1) time and O(1) space**.

The difference is the conversion rule. Implicit conversion is allowed on the **right** operand of a member operator, but **not** on its left operand (the object that calls the function). I tested this:

MEMBER operator+ : 100 + m FAILS

```
friendplus.cpp:10:80: error: no match for 'operator+' (operand types are 'int' and 'Money')
```

FRIEND operator+ : both m + 100 and 100 + m WORK

```
10500 10500      // (m + 100) and (100 + m) both = 105 rupees in paise
```

With a member +, m + 100 works (100 is converted to Money for the argument) but 100 + m does not compile. With a non-member/friend +, both arguments are treated the same, so both forms work. That is why symmetric operators like + are often written as friends or non-members, while += stays a member.

Q5 – Overloading Increment, Comparison and Subscript Operators

Problem: a music player needs time++ and ++time to move one second ahead, comparisons between times, and playlist[i] with a range check.

1. Implementation

playlist.cpp

```cpp
#include <chrono>
#include <iostream>
#include <string>
#include <vector>

class Time {
    int h, m, s;
public:
    Time(int hh = 0, int mm = 0, int ss = 0) : h(hh), m(mm), s(ss) {}
    Time& operator++() {                    // prefix: change, then return the object itself
        if (++s == 60) { s = 0; if (++m == 60) { m = 0; ++h; } }
        return *this;
    }
    Time operator++(int) {                  // postfix: dummy int, return OLD value (a copy)
        Time old = *this;
        ++(*this);
        return old;
    }
    bool operator==(const Time& o) const { return h == o.h && m == o.m && s == o.s; }
    bool operator<(const Time& o) const {
        if (h != o.h) return h < o.h;
        if (m != o.m) return m < o.m;
        return s < o.s;
    }
    friend std::ostream& operator<<(std::ostream& os, const Time& t) {
        return os << t.h << ':' << (t.m < 10 ? "0" : "") << t.m << ':' << (t.s < 10 ? "0" : "") << t.s;
    }
};

class Playlist {
    std::vector<std::string> tracks;
    std::string invalid = "<invalid track>";
public:
    void add(const std::string& t) { tracks.push_back(t); }
    int size() const { return (int)tracks.size(); }

    std::string& operator[](int i) {                    // non-const: allows playlist[i] = "x"
        if (i < 0 || i >= size()) { std::cerr << "Index " << i << " out of range\n"; return invalid; }
        return tracks[i];
    }
    const std::string& operator[](int i) const {        // const: usable on const Playlist
        if (i < 0 || i >= size()) { std::cerr << "Index " << i << " out of range\n"; return invalid; }
        return tracks[i];
    }
};

void printAll(const Playlist& p) {                      // needs the const version
    for (int i = 0; i < p.size(); ++i) std::cout << "  " << i << ": " << p[i] << '\n';
}

int main() {
    Time t(0, 0, 58);
    Time a = t++;     // a gets OLD value
    Time b = ++t;     // b gets NEW value
    std::cout << "a = " << a << "  b = " << b << "  t = " << t << '\n';

    Time r(0, 59, 59); ++r;
    std::cout << "59:59 roll-over -> " << r << '\n';
    Time r2(0, 0, 59); ++r2;
    std::cout << "0:00:59 + 1s -> " << r2 << '\n';
    std::cout << "a < b: " << (a < b) << "   a == b: " << (a == b) << '\n';

    Playlist pl;
    pl.add("Track One"); pl.add("Track Two"); pl.add("Track Three");
    pl[1] = "Track Two (remix)";
    printAll(pl);
    std::cout << "pl[-1]   -> " << pl[-1] << '\n';
    std::cout << "pl[size] -> " << pl[pl.size()] << '\n';

    std::cout << "\nPlayback simulation (song length 0:00:03):\n";
    Time now, end(0, 0, 3);
    while (now < end) { std::cout << "  playing at " << now << '\n'; ++now; }
    std::cout << "  finished at " << now << '\n';
}
```

OUTPUT

```
a = 0:00:58  b = 0:01:00  t = 0:01:00
59:59 roll-over -> 1:00:00
```

```
0:00:59 + 1s -> 0:01:00
a < b: 1   a == b: 0
  0: Track One
  1: Track Two (remix)
  2: Track Three
pl[-1]   -> <invalid track>
pl[size] -> <invalid track>

Playback simulation (song length 0:00:03):
  playing at 0:00:00
  playing at 0:00:01
  playing at 0:00:02
  finished at 0:00:03

(stderr)  Index -1 out of range
(stderr)  Index 3 out of range
```

2. Design & Analysis

How does the compiler tell prefix from postfix ++?

Both are named operator++. The postfix version has an extra unused parameter of type int: operator++(int). When the compiler sees t++ it calls t.operator++(0); for ++t it calls t.operator++(). The int carries no value – it only makes the two signatures different.

Why postfix must return a copy and is slower

t++ has to give back the value from **before** the increment, but the object itself must be incremented. So the function saves the old value in a local object, increments, and returns that saved object **by value**. Returning a reference is impossible (it would refer to a local variable). The extra copy is why postfix can be slower; prefix just increments and returns *this.

Why operator[] needs const and non-const versions

The non-const version returns std::string&, so playlist[1] = "new" can change the track. A const Playlist (for example a function parameter const Playlist&, like printAll) can only call const member functions, and those must not allow modification, so they return const std::string&. Without the const version, printAll would not compile.

What should happen when the index is out of range?

The index has no valid object to return. My version prints an error and returns a dummy string so the program does not crash. In a real project it is better to **throw `std::out_of_range`** (like at()), because a dummy value can hide a bug and let the program continue with wrong data.

3. Experimental Investigation

(a) a = t++ and b = ++t

Starting with t = 0:00:58: Time a = t++; gives a = 0:00:58 (the old value) and t becomes 0:00:59. Then Time b = ++t; makes t = 0:01:00 and b = 0:01:00 (the new value). Output: a = 0:00:58 b = 0:01:00 t = 0:01:00.

(b) Time for 10,000,000 prefix and postfix increments

Optimisation	Prefix ++t	Postfix t++
-O0	21.1 ms	38.0 ms
-O2	6.6 ms	5.1 ms

At **-O0** (no optimisation) postfix is clearly slower (about 1.8 times) because the copy is really made. At **-O2** the compiler inlines the functions and removes the unused copy, so the two are the same and the small difference is just measurement noise. So the cost of postfix is real for large or complicated types and when optimisation is off, but not always visible for a tiny class like Time. (These are my timings; measure on your own machine and repeat a few times.)

(c) Out-of-range access

playlist[-1] and playlist[size] (index 3 with 3 tracks) both printed Index ... out of range and returned <invalid track> – no crash.

(d) Roll-over

0:59:59 + 1 second gave 1:00:00, and 0:00:59 + 1 second gave 0:01:00, so both second-to-minute and minute-to-hour carries work.

4. Conclusion

For class types, ++i is preferred because it does not make a temporary copy. For an int the compiler hides the difference, but for a class (an iterator, a big object) postfix copies the object each time. So when the old value is not needed, write ++i.

5. Predict the Output

counter.cpp

```cpp
#include <iostream>
struct Counter {
    int v;
    Counter& operator++() { ++v; return *this; } // prefix
    Counter operator++(int) { Counter old = *this; ++v; return old; } // postfix
};
int main()
{
    Counter c{5};
    Counter a = c++;
    Counter b = ++c;
    std::cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
}

```

My predicted output (before running):

PREDICTION

```
5 7 7
```

Actual output:

ACTUAL OUTPUT

```
5 7 7
```

They match. Trace: c starts at 5. Counter a = c++; – the postfix version saves the old value (5), increments c to 6 and returns the saved copy, so a.v = 5. Counter b = ++c; – the prefix version increments c to 7 and returns a reference to c, so b.v = 7. Finally c.v = 7.

a and b differ because postfix returns the value **before** the increment, prefix the value **after** it. The **postfix** version (operator++(int)) is the one that makes a copy of the object (Counter old = *this;).

6. Alternative Concept and Complexity Analysis

ALTERNATIVES

```
// (1) at(i) that throws
const std::string& at(int i) const {
    if (i < 0 || i >= size()) throw std::out_of_range("bad track index");
    return tracks[i];
}
// (2) unchecked, like a built-in array
std::string& operator[](int i) { return tracks[i]; }
```

Version (10,000,000 accesses)	-O0	-O2
Checked with range test (used)	36.8 ms	12.6 ms
at() with exception	56.6 ms	12.5 ms
Unchecked []	26.7 ms	16.5 ms

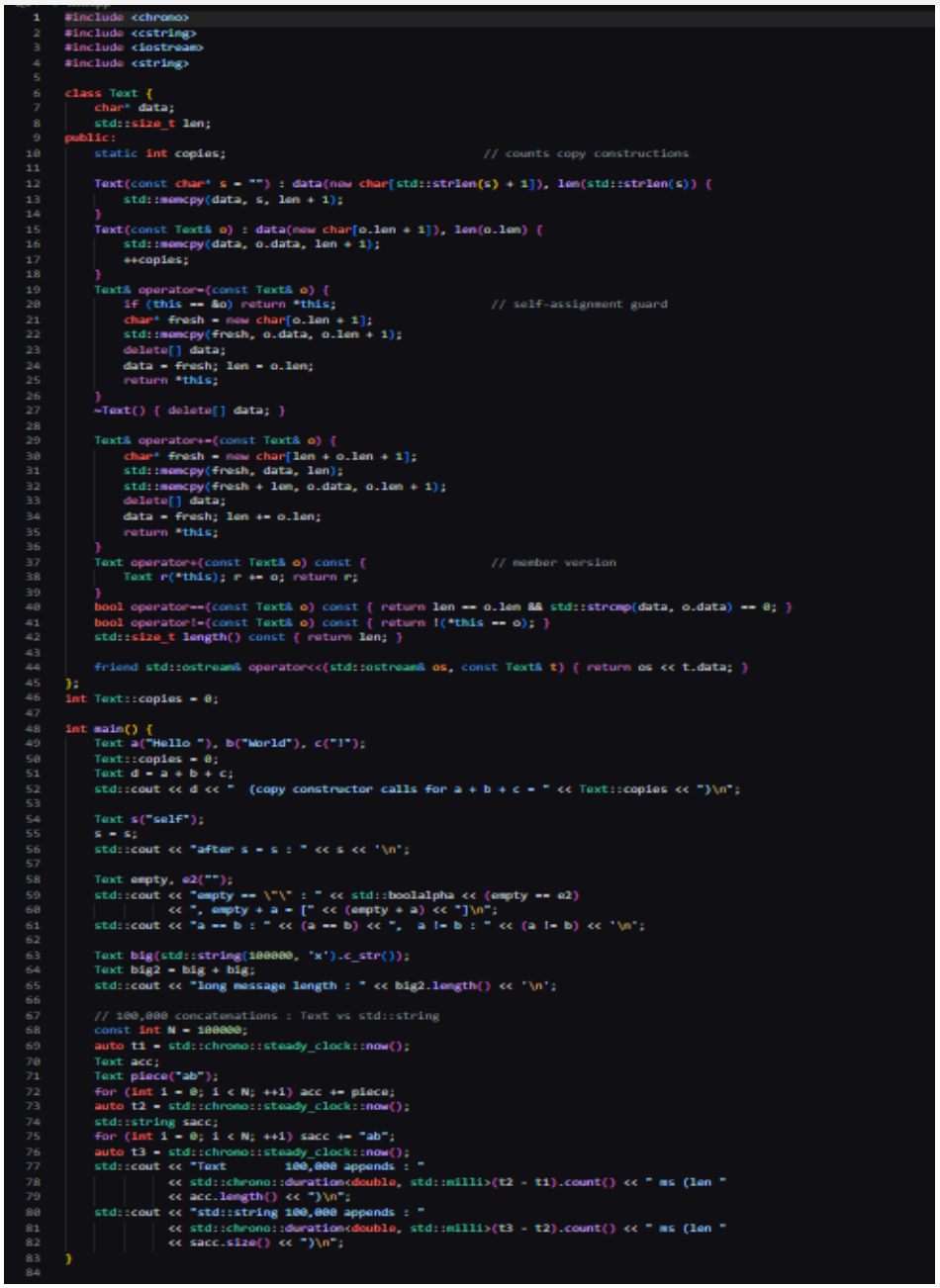
Every version is **O(1) time per access** and O(1) extra space; the checked versions do one extra comparison (two, for both ends of the range). At -O0 the check costs about 10 ms over 10 million accesses (roughly 1 nanosecond per access). At -O2 the three are close together and the order is just noise, so the cost of checking is tiny. Safety is worth it: an unchecked out-of-range read is undefined behaviour. (Timings from my run; repeat on your machine.)

Q6 – The Rules of Operator Overloading

Problem: an SMS gateway uses its own Text class (a dynamically allocated character array) with assignment, +, comparison and printing.

1. Implementation

text.cpp



```cpp
#include <chrono>
#include <cstring>
#include <iostream>
#include <string>

class Text {
    char* data;
    std::size_t len;
public:
    static int copies;                                  // counts copy constructions

    Text(const char* s = "") : data(new char[std::strlen(s) + 1]), len(std::strlen(s)) {
        std::memcpy(data, s, len + 1);
    }
    Text(const Text& o) : data(new char[o.len + 1]), len(o.len) {
        std::memcpy(data, o.data, len + 1);
        ++copies;
    }
    Text& operator=(const Text& o) {
        if (this == &o) return *this;                   // self-assignment guard
        char* fresh = new char[o.len + 1];
        std::memcpy(fresh, o.data, o.len + 1);
        delete[] data;
        data = fresh; len = o.len;
        return *this;
    }
    ~Text() { delete[] data; }

    Text& operator+=(const Text& o) {
        char* fresh = new char[len + o.len + 1];
        std::memcpy(fresh, data, len);
        std::memcpy(fresh + len, o.data, o.len + 1);
        delete[] data;
        data = fresh; len += o.len;
        return *this;
    }
    Text operator+(const Text& o) const {               // member version
        Text r(*this); r += o; return r;
    }
    bool operator==(const Text& o) const { return len == o.len && std::strcmp(data, o.data) == 0; }
    bool operator!=(const Text& o) const { return !(*this == o); }
    std::size_t length() const { return len; }

    friend std::ostream& operator<<(std::ostream& os, const Text& t) { return os << t.data; }
};
int Text::copies = 0;

int main() {
    Text a("Hello "), b("World"), c("!");
    Text::copies = 0;
    Text d = a + b + c;
    std::cout << d << "  (copy constructor calls for a + b + c = " << Text::copies << ")\n";

    Text s("self");
    s = s;
    std::cout << "after s = s : " << s << '\n';

    Text empty, e2("");
    std::cout << "empty == \"\" : " << std::boolalpha << (empty == e2)
              << ", empty + a = [" << (empty + a) << "]\n";
    std::cout << "a == b : " << (a == b) << ",  a != b : " << (a != b) << '\n';

    Text big(std::string(100000, 'x').c_str());
    Text big2 = big + big;
    std::cout << "long message length : " << big2.length() << '\n';

    // 100,000 concatenations : Text vs std::string
    const int N = 100000;
    auto t1 = std::chrono::steady_clock::now();
    Text acc;
    Text piece("ab");
    for (int i = 0; i < N; ++i) acc += piece;
    auto t2 = std::chrono::steady_clock::now();
    std::string sacc;
    for (int i = 0; i < N; ++i) sacc += "ab";
    auto t3 = std::chrono::steady_clock::now();
    std::cout << "Text        100,000 appends : "
              << std::chrono::duration<double, std::milli>(t2 - t1).count() << " ms (len "
              << acc.length() << ")\n";
    std::cout << "std::string 100,000 appends : "
              << std::chrono::duration<double, std::milli>(t3 - t2).count() << " ms (len "
              << sacc.size() << ")\n";
}
```

OUTPUT (-O2; also run with -fsanitize=address, no errors)

```
Hello World!  (copy constructor calls for a + b + c = 2)
after s = s : self
empty == "" : true, empty + a = [Hello ]
a == b : false,  a != b : true
```

```
long message length : 200000
Text        100,000 appends : 530.6 ms (len 200000)
std::string 100,000 appends : 0.43 ms (len 200000)
```

2. Design & Analysis

Operators that cannot be overloaded

Operator	Why it cannot be overloaded
:: (scope resolution)	Its operands are names of classes/namespaces, not values, so there is no object to pass to a function.
. (member access)	The right side is a member name, not an expression. Changing it would change the meaning of every member access in the language.
.* (pointer-to-member access)	Same reason as .; it is a core language access mechanism.
?: (conditional)	It takes three operands and evaluates only one of the two branches; a function call would evaluate all arguments.
sizeof	It is evaluated by the compiler at compile time and works on types as well as values.

The rules of operator overloading

- You cannot invent new operators (for example **).
- **Arity** (number of operands), **precedence** and **associativity** stay the same as for the built-in operator. a + b * c still does * first.
- At least one operand must be a user-defined type (class or enum). You cannot change int + int.
- =, [], () and -> must be member functions.
- Overloaded operators should keep their natural meaning (do not make + subtract).

Why operator= must be a member

The language requires it. The left side of an assignment has to be an object of the class itself, and the compiler already generates a copy assignment member for every class; allowing a free function operator= would conflict with it. Keeping it a member guarantees the left operand is the actual object being assigned to.

When should an overloaded operator be a friend?

- When the left operand is **not** your class (<<, >> with streams).
- When you need symmetric conversions on both operands (100 + m and m + 100).
- When it needs access to private data and a public getter would be awkward. A friend should be used sparingly.

3. Experimental Investigation

(a) Copies made when a + b + c is evaluated

The counter showed **2** copy-constructor calls for a + b + c. Reading it: (a + b) makes a copy of a inside operator+ (1); the returned temporary is then the left operand of + c, and that call copies it again (2). The returned objects themselves are not copied because the compiler removes that copy (return value optimisation). With optimisation off or an older compiler the count can be higher.

(b) Address sanitizer

The program compiled with -fsanitize=address runs with **no errors**. During development the compiler warning -Wreorder appeared (members data and len were initialised in a different order from their declaration). It was harmless here because the code did not depend on the order, but I changed the initializer list to follow the declaration order, and the warning disappeared.

(c) Text vs std::string – 100,000 concatenations (appending "ab")

Class	Time (-O2)	Final length
Text (own class)	530.6 ms	200,000
std::string	0.43 ms	200,000

std::string was about **1,200 times faster**. The reason: my Text::operator+= allocates a new array of exactly the new size and copies the whole old text every time, so each append costs O(current length) and 100,000 appends cost **$O(n^2)$** in total. std::string keeps spare capacity and grows geometrically (roughly doubling), so appends are **amortised O(1)**. The fix for Text would be a separate capacity member with the same growth rule. (Timings from my run; yours will differ but the huge gap will remain.)

(d) Assigning a Text to itself

s = s; left the text unchanged (self). The check if (this == &o) return *this; prevents deleting the array we are about to copy from. I also build the new array before deleting the old one.

Other tests: an empty Text and "" compare equal, empty + a gives "Hello ", and a 100,000-character message concatenated with itself gave a 200,000-character Text without errors.

4. Conclusion

Writing Text showed me how many things must be right to own memory: the Rule of Three, self-assignment, exception safety, and growth strategy. A simple-looking class that works for small tests can still be hundreds of times slower than the standard one. In real programs I would use std::string. I would still write such a class to **learn** how strings work, when I must control memory exactly (embedded systems, fixed buffers, custom allocators), or when I need an unusual feature the standard class does not have.

5. Alternative Concept and Complexity Analysis

ALTERNATIVES

```
// operator+ written with +=   (first parameter by value)
Text operator+(Text a, const Text& b) { a += b; return a; }


// or simply
std::string r = s1 + s2 + s3;
```

Let the lengths be n1, n2, n3. Counting characters copied:

Version	a + b	(a + b) + c	Total copied	Temporary space
Member operator+ (mine): copy *this, then +=	n1 + (n1+n2)	(n1+n2) + (n1+n2+n3)	about 4·n1 + 3·n2 + n3 → O(n1+n2+n3)	copy + new array alive together, up to ≈ 2·(n1+n2)
operator+(Text a, const Text& b) with +=	copy a (n1), += copies n1+n2	the temporary is used directly as the parameter (C++17 copy elision), then += copies (n1+n2)+n3	about 3·n1 + 2·n2 + n3 → O(n1+n2+n3)	one parameter copy + one new array at a time

Version	a + b	(a + b) + c	Total copied	Temporary space
std::string	n1 + n2	only n3 appended to the temporary (amortised)	about n1 + n2 + n3 → O(n1+n2+n3)	one temporary of n1+n2 (+ spare capacity)

All versions are **O(n1+n2+n3)** time and space, but the constant is different: my first design copies each character several times, while std::string copies each character about once (and can reuse the temporary). Writing operator+ with += is a good habit because the logic stays in one place, and passing the left operand by value lets the compiler avoid a copy when it is already a temporary.

Q7 – Design a Complex-Number Class from First Principles

Problem: an AC impedance calculator. Resistor: R + 0j, inductor: 0 + jωL, capacitor: 0 − j/(ωC). Series impedances add; parallel impedances combine as (Z1 × Z2) / (Z1 + Z2). It must print rectangular form and magnitude and handle division by zero safely.

Stage 1 – Interpret the Problem

Data and operations

- **Data:** a real part and an imaginary part, both double (impedance in ohms).
- **Operations:** add (series), multiply and divide (parallel formula), subtract, compare, print as a + bj, and find the magnitude $|Z| = \sqrt{(a^2 + b^2)}$.

Constructors needed

Constructor	Needed for
Complex()	an empty value 0 + 0j (arrays, accumulating a sum)
Complex(double r)	a pure resistor R + 0j; also allows mixing with ordinary numbers
Complex(double r, double i)	inductors, capacitors and results in rectangular form

Requirements as measurable statements

No.	Requirement	How it is checked
R1	Complex can be created with 0, 1 or 2 arguments.	Compile and print each form.
R2	+, −, ×, ÷ agree with hand calculation to within 1e-9.	(3+4j)(1−2j) = 11 − 2j and (3+4j)/(1−2j) = −1 + 2j.
R3	Series RLC at 50 Hz with R = 10 Ω, L = 0.1 H, C = 100 µF gives 10 − 0.415j Ω.	Compare with the hand calculation.
R4	Parallel RLC gives 10.000 + 0.042j Ω (|Z| ≈ 10.000 Ω).	Hand calculation using 1/Z = 1/R + 1/jXL + 1/(−jXC).
R5	Dividing by 0 + 0j does not crash; it reports an error.	Catch the exception.
R6	Output format is "a + bj" with 3 decimals.	Visual check.

Stage 2 – Design

CLASS DESIGN

```
class Complex {
   double re, im;                          // private data
public:
   Complex();  Complex(double r);  Complex(double r, double i);
   double real() const;  double imag() const;  double magnitude() const;
   Complex operator+(const Complex&) const;
   Complex operator-(const Complex&) const;
   Complex operator*(const Complex&) const;
   Complex operator/(const Complex&) const;   // throws on zero divisor
   bool    operator==(const Complex&) const;  // uses tolerance 1e-9
   friend std::ostream& operator<<(std::ostream&, const Complex&);
};
```

Operator	Formula (a = x1 + j y1, b = x2 + j y2)	Member / friend
a + b	(x1 + x2) + j(y1 + y2)	Member
a − b	(x1 − x2) + j(y1 − y2)	Member
a × b	(x1·x2 − y1·y2) + j(x1·y2 + y1·x2)	Member
a ÷ b	$d = x2^2 + y2^2$; (x1·x2 + y1·y2)/d + j(y1·x2 − x1·y2)/d (error if d = 0)	Member
a == b	|x1 − x2| < 1e-9 and |y1 − y2| < 1e-9	Member
os << a	prints x ± yj	Friend (left operand is the stream)
|a|	$\sqrt{(x^2 + y^2)}$	Member function magnitude()

Members are used for the arithmetic operators because the left operand is always a Complex in the calculator. << must be a non-member friend, as explained in Q4.

Stage 3 – Implement and Verify

complex.cpp

```cpp
#include <cmath>
#include <complex>
#include <iomanip>
#include <iostream>
#include <stdexcept>

class Complex {
    double re, im;
public:
    Complex() : re(0), im(0) {}                         // default
    Complex(double r) : re(r), im(0) {}                 // from real part (resistor); allows 100 + z
    Complex(double r, double i) : re(r), im(i) {}       // rectangular form

    double real() const { return re; }
    double imag() const { return im; }
    double magnitude() const { return std::sqrt(re * re + im * im); }

    Complex operator+(const Complex& o) const { return Complex(re + o.re, im + o.im); }
    Complex operator-(const Complex& o) const { return Complex(re - o.re, im - o.im); }
    Complex operator*(const Complex& o) const {
        return Complex(re * o.re - im * o.im, re * o.im + im * o.re);
    }
    Complex operator/(const Complex& o) const {
        double d = o.re * o.re + o.im * o.im;
        if (d == 0.0) throw std::domain_error("division by zero impedance");
        return Complex((re * o.re + im * o.im) / d, (im * o.re - re * o.im) / d);
    }
    bool operator==(const Complex& o) const {
        const double eps = 1e-9;
        return std::fabs(re - o.re) < eps && std::fabs(im - o.im) < eps;
    }
    friend std::ostream& operator<<(std::ostream& os, const Complex& z) {
        return os << std::fixed << std::setprecision(3) << z.re
                  << (z.im < 0 ? " - " : " + ") << std::fabs(z.im) << "j";
    }
};

// helpers for circuit elements
Complex resistor(double R)              { return Complex(R, 0); }
Complex inductor(double w, double L)    { return Complex(0, w * L); }
Complex capacitor(double w, double C)   { return Complex(0, -1.0 / (w * C)); }
Complex parallel(const Complex& a, const Complex& b) { return (a * b) / (a + b); }

int main() {
    const double PI = 3.14159265358979323846;
    double w = 2 * PI * 50;                         // 50 Hz supply
    Complex R = resistor(10), L = inductor(w, 0.1), C = capacitor(w, 100e-6);

    std::cout << "R = " << R << "\nXL = " << L << "\nXC = " << C << "\n\n";

    Complex series = R + L + C;                     // series RLC
    std::cout << "Series RLC   Z = " << series << "   |Z| = " << series.magnitude() << " ohm\n";

    Complex par = parallel(parallel(R, L), C);      // parallel RLC
    std::cout << "Parallel RLC Z = " << par << "   |Z| = " << par.magnitude() << " ohm\n";

    // hand-check with a simple case: (3+4j)*(1-2j) = 11 - 2j ; (3+4j)/(1-2j) = -1 + 2j
    std::cout << "\nHand check: (3+4j)*(1-2j) = " << Complex(3, 4) * Complex(3 * 0 + 1, -2)
              << " ,  (3+4j)/(1-2j) = " << Complex(3, 4) / Complex(1, -2) << '\n';
    std::cout << "Mixed: 100 + (0+5j) = " << Complex(100) + Complex(0, 5) << '\n';
    std::cout << "(3+4j) == (3+4j) : " << std::boolalpha << (Complex(3, 4) == Complex(3, 4)) << '\n';

    try { std::cout << R / Complex(0, 0) << '\n'; }
    catch (const std::domain_error& e) { std::cout << "Caught: " << e.what() << '\n'; }

    // std::complex comparison
    std::complex<double> zc(3, 4), zd(1, -2);
    std::cout << "std::complex (3+4j)/(1-2j) = " << zc / zd << '\n';
    std::complex<double> inf = std::complex<double>(10, 0) / std::complex<double>(0, 0);
    std::cout << "std::complex 10/0 = " << inf << "  (no exception, gives nan/inf)\n";
}
```

OUTPUT

R = 10.000 + 0.000j

XL = 0.000 + 31.416j

XC = 0.000 - 31.831j

Series RLC Z = 10.000 - 0.415j |Z| = 10.009 ohm

Parallel RLC Z = 10.000 + 0.042j |Z| = 10.000 ohm

```
Hand check: (3+4j)*(1-2j) = 11.000 - 2.000j ,  (3+4j)/(1-2j) = -1.000 + 2.000j
Mixed: 100 + (0+5j) = 100.000 + 5.000j
(3+4j) == (3+4j) : true
Caught: division by zero impedance
std::complex (3+4j)/(1-2j) = (-1.000,2.000)
std::complex 10/0 = (inf,-nan)  (no exception, gives nan/inf)
```

Verification by hand calculation

$\omega = 2\pi \times 50 = 314.159$ rad/s.

- $XL = \omega L = 314.159 \times 0.1 =$ **31.416 Ω**
- $XC = 1/(\omega C) = 1 / (314.159 \times 100\times10^{-6}) =$ **31.831 Ω**, so the capacitor is $0 - 31.831j$
- **Series:** $Z = 10 + j(31.416 - 31.831) =$ **10 − 0.415j Ω**, $|Z| = \sqrt{(100 + 0.172)} =$ **10.009 Ω** – matches the program.
- **Parallel:** $Y = 1/10 + 1/(j31.416) + 1/(-j31.831) = 0.1 - j0.031831 + j0.031416 = 0.1 - j0.000415$ S. $Z = 1/Y = (0.1 + j0.000415)/(0.1^2 + 0.000415^2) =$ **9.9998 + 0.0415j Ω**, $|Z| \approx 10.000$ Ω – matches (the program prints 10.000 + 0.042j after rounding to 3 decimals). I also cross-checked both results with an independent Python calculation.
- **Multiply/divide check:** $(3+4j)(1-2j) = 3 - 6j + 4j - 8j^2 = 3 - 2j + 8 =$ **11 − 2j**. $(3+4j)/(1-2j) = (3+4j)(1+2j)/5 = (3 + 6j + 4j - 8)/5 =$ **−1 + 2j**. Both match.

Division by a zero impedance

R / Complex(0, 0) made the denominator d = 0, so operator/ threw std::domain_error, which was caught and printed Caught: division by zero impedance. The program did not crash or print nan.

Stage 4 – Defend the Design

Comparison with std::complex<double>

Point	My Complex	std::complex<double>
Operators	+ − × ÷ == <<	All of those plus +=, −=, unary −, polar form, abs, arg, conj, exp, sqrt, trig
Division by zero	Throws an exception (explicit)	No exception; gives inf/nan (shown above: (inf,−nan))

Point	My Complex	std::complex<double>
Division accuracy	Simple formula; can overflow for huge values	Library code handles overflow/underflow better
Testing	Only my tests	Standard, widely tested and optimised
Printing	"10.000 − 0.415j"	"(10,-0.415)"

Why each constructor and operator is needed

- Default constructor: arrays and running totals. One-argument constructor: resistors and mixing with plain numbers. Two-argument constructor: inductors, capacitors and results.
- + for series impedances; * and / for the parallel formula; − for differences (for example voltage drops); == for tests (with a tolerance because doubles are not exact); << to print results; magnitude() because the lab needs |Z|.

One remaining limitation

Because +, −, ×, ÷ are members, **`100 + z` does not compile** (only z + 100 does; see Q4). Making them non-member functions would fix this. Other limits: no +=, no unary minus, and the tolerance in == (1e-9) is fixed and is not a scaled/relative tolerance.

Alternative Concept and Complexity Analysis

Alternative: use std::complex<double> and add a small safe-divide function for the zero check.

ALTERNATIVE

```
#include <complex>
using Z = std::complex<double>;

Z safeDivide(Z a, Z b) {
   if (std::norm(b) == 0.0) throw std::domain_error("division by zero impedance");
   return a / b;
}
Z parallel(Z a, Z b) { return safeDivide(a * b, a + b); }
```

	My Complex	std::complex<double>
Time per operation	O(1)	O(1)

	My Complex	std::complex<double>
Space per object	2 doubles (16 bytes)	2 doubles (16 bytes)
Accuracy	Good for normal values; naive division formula	Same or better; careful handling of overflow and special values
Work for division by zero	Already built in (throws)	Must add a check yourself, otherwise silent inf/nan
Code to write	About 40 lines	Almost none

Both are O(1) time and use two doubles. In a real project I would use std::complex<double> with a small safe-divide wrapper, because it is tested and complete. Writing my own class was useful to understand how operators, constructors and friends fit together.

Submission Checklist (GitHub and README)

Folder layout of the repository https://github.com/Hemalatha-1806/DSA0102-C-

LAYOUT

```
DSA01-<192524475>-Assignment3/
 README.md
 Q1/  book.cpp  reorder.cpp  noDefault.cpp  explicit.cpp  defaultargs.cpp  screenshots/
 Q2/  shallow.cpp  deep.cpp  experiments.cpp  screenshots/
 Q3/  auditorium.cpp  leak.cpp  predict.cpp  screenshots/
 Q4/  money.cpp  tests.cpp  screenshots/
 Q5/  playlist.cpp  counter.cpp  bench.cpp  screenshots/
 Q6/  text.cpp  screenshots/
 Q7/  complex.cpp  screenshots/
```

Commands to compile each type of program:

COMMANDS

```
g++ -std=c++17 -Wall -Wextra file.cpp -o file          # normal
g++ -std=c++17 -g -fsanitize=address file.cpp -o file    # sanitizer runs (Q2, Q3, Q6)
g++ -std=c++17 -O0 bench.cpp -o bench0                  # timing at -O0 (Q5)
g++ -std=c++17 -O2 bench.cpp -o bench2                  # timing at -O2 (Q5)
```

Your README.md should contain: your name, register number, compiler and version (for example g++ 13), the exact compile command, the steps to run each program, and one or two lines of result for each task. Make **at least one commit per task**, for example:

GIT

```
git init
git add Q1 && git commit -m "Q1: constructors and delegation, experiments"
git add Q2 && git commit -m "Q2: shallow vs deep copy, sanitizer reports"
...
git remote add origin <your repo URL>
git push -u origin main
```